Class 8: Experiment Tracking and Model Tuning

Data-driven Systems Engineering

Agenda

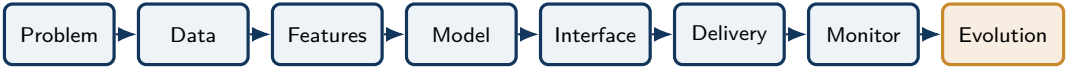
- Why experiment tracking is part of engineering, not only analysis.
- Designing experiments before running them.
- What must be logged to compare models fairly.
- Tuning strategies and when to use each one.
- Repository context: Neptune-style experimentation and fraud-model comparison.
- How experiment records support MLOps later.

Why this class fills an important gap

Learning goals

- Turn model comparison into a reproducible engineering practice.
- Connect hyperparameter tuning with logging, provenance, and fair comparison.
- Prepare students for MLOps workflows where experiments become auditable assets.
- Distinguish exploration from disciplined experimentation.

Course Map and Running Cases



Today in the course

Focus: Disciplined experimentation and fair model comparison.
Bridge: This class connects modeling ideas to engineering evidence by making runs, baselines, and provenance comparable.

Fraud case

In the fraud case, experiment tracking supports fair comparison across preprocessing choices, model families, and threshold strategies.

Pasteurization case

In the pasteurization case, experiment tracking can compare forecasting variants, online settings, or drift-handling choices without losing context.

Course thread: The thread here is that a good model choice must be explainable, repeatable, and ready for later delivery.

An experiment is a decision-making tool

- The purpose of experimentation is not to collect many scores, but to support a credible model choice.
- Every run should answer a question: does a new feature set help, does a different model family help, or does tuning justify extra complexity?
- When the question is unclear, experiment tracking becomes a storage system for confusion.

Design the experiment before pressing run

- Define the target metric and why it matters operationally.
- Fix the evaluation protocol before looking at results.
- Decide which variables are changing and which are held constant.
- Record what counts as a meaningful improvement.

Examples

Changing both preprocessing and model family in the same run may be useful, but it makes causal interpretation harder unless the experiment question was framed that way.

What must be tracked in every experiment

- Dataset version or extraction date.
- Feature pipeline version.
- Hyperparameters and random seeds.
- Metrics, confusion matrices, and artifacts.
- Code commit or notebook snapshot.

Artifacts, lineage, and reproducibility

- Metrics alone are insufficient because teams also need the trained artifact, code context, and preprocessing lineage.
- A useful run can be replayed, audited, and compared months later.
- Lineage means we can answer: which data, which code, which parameters, and which artifact produced this result?

Why experiment tracking matters

- Without metadata, good results become hard to reproduce.
- Teams need evidence for why one model was selected over another.
- Tracking helps separate real improvements from accidental differences in preprocessing or data splits.
- Experiment history becomes valuable once retraining and audits begin.

Repo anchor for this lecture

Repo activity

‘notebooks/Class7_Model_Tuning_Neptune.ipynb‘ gives a natural entry point for discussing experiment metadata, model comparison, and why ad hoc notebook tuning quickly becomes unmanageable.

Better example

Compare two fraud models with similar AUC but different precision at the operational review threshold.

What makes a comparison fair?

- Same dataset split or same evaluation protocol.
- Same preprocessing assumptions unless preprocessing is the experiment itself.
- Metrics chosen to reflect the operational decision.
- Clear record of training time, latency, artifact size, and maintenance burden.

Common sources of experimental bias

- Reusing the test set repeatedly until it becomes part of the tuning loop.
- Comparing runs produced from different random splits without documenting variance.
- Mixing notebook state, manual edits, and hidden preprocessing changes.
- Reporting only the winning run instead of the search process that produced it.

Model search strategies

Strategy	Best use
Manual tuning	Small problems and pedagogical intuition.
Grid search	Small, well-structured search spaces.
Random search	Better coverage when many hyperparameters matter unevenly.
Bayesian optimization	Expensive models where each run has high cost.

Baselines matter more than they seem

- A tuned model should beat a simple baseline by a meaningful margin.
- Baselines expose whether the added complexity is actually buying value.
- They also help students avoid treating hyperparameter search as the first step.

Examples

Linear regression, a default tree-based model, or a previously deployed model can all serve as strong baselines depending on the task.

Notebook example: defining a search space

```
model = GradientBoostingRegressor(random_state=42)

param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "min_samples_split": [2, 5, 10],
}
```

- This follows ‘notebooks/Class7_Model_Tuning_Neptune.ipynb‘.
- It gives students a realistic example of trade-offs among model complexity, fit time, and predictive gain.

Notebook example: tuning and logging

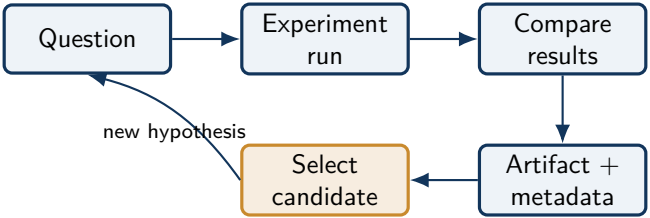
```
grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1,
)

grid_search.fit(X_train, y_train)
run["best/parameters"] = grid_search.best_params_
run["best/cv_score"] = grid_search.best_score_
```

- The important idea is not Neptune alone, but the discipline of storing parameters and metrics together.
- That same pattern later maps naturally to MLflow, Weights & Biases, or internal experiment registries.

From tuning to model selection

- The best validation score is not automatically the best system choice.
- Sometimes a slightly weaker model is easier to deploy, explain, or monitor.
- The selected model should align with cost, interpretability, and retraining needs.

From notebook run to governed asset



Common tuning mistakes

- Tuning on the test set.
- Comparing runs trained on different preprocessing logic.
- Reporting only the best metric, not variability or cost.
- Ignoring class imbalance and threshold behavior.
- Forgetting that a slightly worse model may be much easier to operate.

What this prepares us for next

- Delivery pipelines need stable artifacts and reproducible build inputs.
- Requirements engineering needs measurable acceptance criteria, not only better scores.
- Monitoring later depends on knowing which experiment produced the deployed behavior.

Notebook example: evaluate and persist the winner

```
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

run["test/mse"] = mean_squared_error(y_test, y_pred)
run["test/r2"] = r2_score(y_test, y_pred)

joblib.dump(best_model, "best_gb_model.pkl")
run["artifacts/model"].upload("best_gb_model.pkl")
```

- A useful experiment ends with a traceable artifact, not only with a screenshot of the best score.
- This slide also helps bridge experiment tracking to model registry and deployment topics.

Technology links

Papers and materials

Neptune documentation; MLflow documentation; scikit-learn model selection guide; GridSearchCV API.

From This Class to the Next

What stays with us

- A useful experiment records data, configuration, metrics, and artifacts together.
- Hyperparameter search is valuable only when it is reproducible and comparable.
- Tracking prevents teams from repeating work or trusting results they cannot explain.

Project use

This gives project teams a disciplined way to compare alternatives, preserve evidence, and package the winning model for later delivery.

Next connection

Class 9 extends that traceability into containers and CI/CD pipelines so experiments can become repeatable releases.

Course thread: Reproducible experimentation is the bridge between exploratory modeling and operational delivery.

Papers and materials

Papers and materials

Bergstra and Bengio (2012), Random Search; Bergstra et al. (2013), Hyperopt; Neptune documentation; MLflow documentation.